

Sri Lanka Institute of Information Technology



IT3060

Human Computer Interaction

Lab 5- Activity 1

Frontend Technology Comparison for FitFlow Redesign

IT23657878- W.R.S.D.Diwyanjali

1. Introduction

FitFlow is a fitness tracking application that experienced declining user retention and a decrease in app-store ratings. The redesign initiative was started to identify user pain points and introduce features such as AI-powered personalized workout plans, social community features and improved nutrition tracking. The case study describes a human-centred redesign process and identifies the need for a high-performance, privacy-focused mobile experience.

The case study states that React Native was selected for the redesigned mobile frontend because of its cross-platform efficiency, rapid development capabilities and support for complex fitness-related animations. The following comparison evaluates React Native alongside Flutter, Kotlin Multiplatform and Swift/SwiftUI against the requirements specified in Lab Exercise 05.

2. FitFlow Frontend Requirements

- The frontend technology should support the following FitFlow requirements:
- Seamless experiences across iOS and Android, with consideration for web compatibility.
- High performance for workout experiences and complex animations.
- Rapid development and efficient delivery for a mid-sized health-tech startup.
- Integration with AI-powered personalized workout features.
- Support for real-time social interactions and notifications.
- Support for camera-based nutrition tracking and computer-vision features.
- Strong security and privacy considerations for wellness-related user data.
- Maintainability and reasonable long-term development cost.

3. Flutter

Flutter is a cross-platform UI framework that enables developers to build applications for multiple platforms from a shared codebase.

- Strengths: High code reuse across supported platforms and a single UI development approach.
- Strengths: Strong capability for custom interfaces, animations and responsive user experiences.
- Strengths: Suitable for rapid cross-platform development.
- Weaknesses: Some platform-specific capabilities may require native integrations or additional packages.
- Weaknesses: Teams must consider framework/plugin maintenance when depending on third-party packages.

FitFlow relevance: Flutter can support the visually rich workout interface and cross-platform requirement, but the team would need to validate the required AI, camera and platform integrations.

4. React Native

React Native is a cross-platform mobile framework based on React that allows a shared application codebase to target mobile platforms.

- Strengths: High code reuse between iOS and Android.
- Strengths: Rapid development and a mature JavaScript/TypeScript ecosystem.
- Strengths: Good fit for interactive mobile interfaces and complex animations.
- Strengths: Can integrate with native modules when platform-specific functionality is required.
- Weaknesses: Some advanced device or platform-specific features may require native code.

FitFlow relevance: The case study explicitly selected React Native because of cross-platform efficiency, rapid development and support for complex fitness animations.

5. Kotlin Multiplatform

Kotlin Multiplatform allows teams to share selected application logic across platforms while retaining the ability to create platform-specific UI and integrations.

- Strengths: Strong code sharing for business logic while preserving native platform capabilities.
- Strengths: Kotlin provides strong typing and good maintainability for larger codebases.
- Strengths: Useful when a product needs deeper native integration.
- Weaknesses: UI sharing and project architecture can be more complex than a single cross-platform UI approach.
- Weaknesses: Teams need sufficient Kotlin and native-platform knowledge.

FitFlow relevance: It is a strong option where native performance and platform-specific capabilities are priorities, but it may require more platform-specific expertise.

6. Swift / SwiftUI

Swift and SwiftUI are Apple technologies designed for building applications for Apple's platforms.

- Strengths: Strong native performance and close integration with iOS capabilities.
- Strengths: Excellent access to Apple platform APIs, device capabilities and accessibility features.
- Strengths: SwiftUI provides a modern declarative approach to Apple UI development.
- Weaknesses: Primarily an Apple-platform solution and therefore does not provide the same shared-code approach for Android.
- Weaknesses: A separate Android implementation would be required for a full iOS and Android product.

FitFlow relevance: It is suitable for a premium iOS experience but is less aligned with FitFlow's requirement for an efficient cross-platform mobile strategy.

7. Frontend Technology Comparison

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift/SwiftUI
Development speed	High	High	Medium	High for Apple platforms
Code reusability	High across platforms	High across mobile platforms	High for shared logic; UI depends on approach	Low for Android; mainly Apple
Performance	High	High; native integration available	High with native platform capabilities	Very high on Apple platforms
Ecosystem support	Large cross-platform ecosystem	Large JavaScript/React ecosystem	Strong Kotlin/native ecosystem	Strong Apple ecosystem
Learning curve	Medium	Medium	Medium–High	Medium for Apple development
Web compatibility	Available	Web option exists but needs validation for project needs	Web support depends on architecture/target	Not a cross-platform web solution
AI/ML integration	Can integrate through platform/ML libraries	Can integrate through native modules and services	Strong native integration potential	Strong Apple ML integration
Real-time features	Well supported through APIs/SDKs	Well supported through Firebase and other services	Supported through shared logic/native integrations	Supported through Apple/networking frameworks
Maintenance cost	Moderate	Moderate	Moderate–High depending on native code	High for a cross-platform product because Android needs another implementation

Security	Can use platform security mechanisms	Can use native security capabilities and secure services	Strong native security integration	Strong Apple security integration
----------	--------------------------------------	--	------------------------------------	-----------------------------------

Note: The comparison above is a qualitative assessment for the FitFlow scenario. The lab sheet requires these criteria to be considered together with FitFlow's specific requirements rather than treating one criterion as sufficient on its own.

8. Suitability for the FitFlow Redesign

FitFlow requires a mobile frontend that can deliver a consistent user experience on iOS and Android, support fast iteration, handle rich fitness interactions, and integrate with AI, real-time social features and device capabilities such as camera-based nutrition tracking.

- Flutter
- Provides strong cross-platform development and can support highly customised fitness interfaces.
- It is a viable choice when the team wants a single UI technology across mobile platforms.
- React Native
- Provides strong cross-platform code reuse and rapid development.
- Fits well with real-time services, JavaScript/React development and native integrations.
- Most importantly, it matches the technology choice documented in the FitFlow case study.
- Kotlin Multiplatform
- Provides a strong option when sharing business logic while retaining native platform control is important.
- It can be attractive for teams with strong Kotlin and native development expertise.
- Swift/SwiftUI
- Provides excellent native iOS capabilities and performance.
- However, FitFlow's cross-platform requirement means an additional Android implementation would be needed.

9. Recommendation

Based on the FitFlow case study and the criteria in the lab sheet, React Native is recommended as the frontend technology for the FitFlow redesign.

Justification:

Cross-platform development: A shared mobile codebase supports efficient delivery for both iOS and Android.

Development speed: The case study identifies rapid development as an important startup requirement.

Performance and interaction: The case study specifically notes support for complex animations required in fitness experiences.

Integration: React Native can work with backend services, real-time services and native device capabilities required by FitFlow.

AI and nutrition features: The frontend can connect to AI services and camera/computer-vision capabilities through appropriate service or native integrations.

Maintainability: Shared application code can reduce duplication compared with maintaining separate iOS and Android applications.

Case-study alignment: The selected stack in the case study is React Native with Node.js, Express and Firebase, making React Native the option most directly supported by the supplied scenario.

10. Conclusion

Flutter, React Native, Kotlin Multiplatform and Swift/SwiftUI can all support modern mobile application development, but they provide different trade-offs between cross-platform reuse, native control, development speed and platform coverage. For FitFlow, the need for efficient iOS/Android development, high-performance fitness interactions, AI and real-time integrations, and a maintainable codebase makes React Native a suitable choice. This conclusion is also consistent with the technology selection explicitly described in the supplied FitFlow case study.

11. Source Basis

Primary source: IT3060 Human Computer Interaction Semester 2 2026 – FitFlow Redesign Case Study and Lab Exercise 05. The FitFlow-specific requirements and the documented React Native selection are taken from the supplied case study; the comparative descriptions are presented to satisfy Activity 1's required technology comparison criteria.